CMS — Production Refactoring Plan

Tech Lead Review | Laravel 13 · PHP 8.3 · Clinic Management API

Executive Summary

The repository is a Laravel 13 API for a clinic management system. The foundation is solid — modern PHP 8.3, proper Eloquent models, enums, and IDE helpers in place. However, there are **critical architectural flaws** that will cause security failures, runtime conflicts, and unmaintainability at production scale. This plan organizes all issues by severity and provides concrete implementation code for every fix.

🔴 CRITICAL — Fix Before Any Deployment

---

1. Triple Authentication Conflict (Security Blocker)

What was found: The codebase has THREE authentication systems installed simultaneously:

- `laravel/passport` (OAuth2 server) in `composer.json`
- `php-open-source-saver/jwt-auth` (JWT stateless tokens) — `User` implements `JWTSubject`
- Laravel Sanctum — `User` has `PersonalAccessToken` tokens via IDE helper

This creates unpredictable middleware behavior, token validation conflicts, and massively inflated attack surface. A production security audit will flag this immediately.

Fix: Pick ONE. For a stateless API, **JWT is the right choice**. Remove Passport and Sanctum.

```
composer remove laravel/passport
# Remove Sanctum from User model and config/app.php
```

`app/Models/User.php` — clean version:

```
use PHPOpenSourceSaver\JWTAuth\Contracts\JWTSubject;

class User extends Authenticatable implements JWTSubject
{
    public function getJWTIdentifier(): mixed
    {
        return $this->getKey();
    }

    public function getJWTCustomClaims(): array
    {
        return [
            'role'     => $this->role,
            'clinic_id' => $this->clinic_id,
        ];
    }
}
```

config/auth.php：

```
'guards' => [
    'api' => [
        'driver'   => 'jwt',
        'provider' => 'users',
    ],
],
```

---

2. Custom `RefreshToken` Model (Reinventing the Wheel)

What was found: The User model has a `refreshTokens` relationship pointing to a custom `RefreshToken` model. JWT-auth handles refresh tokens natively via `auth()->refresh()`. This custom model is dead weight and a security liability — it won't be invalidated properly on logout.

Fix: Delete `app/Models/RefreshToken.php` and its migration. Use JWT-auth's built-in refresh:

```
// AuthController.php
public function refresh(): JsonResponse
{
    return $this->respondWithToken(auth()->refresh());
}
```

```
public function logout(): JsonResponse
{
    auth()->invalidate(true); // blacklists the current token
    return response()->json(['message' => 'Successfully logged out']);
}
```

Enable the JWT blacklist in `config/jwt.php`:

```
'blacklist_enabled' => env('JWT_BLACKLIST_ENABLED', true),
'blacklist_grace_period' => env('JWT_BLACKLIST_GRACE_PERIOD', 0),
```

---

3. No Role-Based Access Control (RBAC)

What was found: The `User` model has no `role` field or permission system. There is no separation between Admin, Doctor, and Receptionist access. Any authenticated user can presumably call any endpoint.

Fix: Install `spatie/laravel-permission` and define roles clearly.

```
composer require spatie/laravel-permission
php artisan vendor:publish --provider="Spatie\Permission\PermissionServiceProvide
php artisan migrate
```

`database/seeders/RoleSeeder.php`:

```
use Spatie\Permission\Models\Role;
use Spatie\Permission\Models\Permission;

Role::create(['name' => 'admin']);
Role::create(['name' => 'doctor']);
Role::create(['name' => 'receptionist']);

// Assign fine-grained permissions
Permission::create(['name' => 'manage appointments']);
Permission::create(['name' => 'write prescriptions']);
Permission::create(['name' => 'manage patients']);
Permission::create(['name' => 'manage clinic']);
```

In routes:

```
Route::middleware(['auth:api', 'role:doctor'])->group(function () {
    Route::apiResource('prescriptions', PrescriptionController::class);
```

```
});

Route::middleware(['auth:api', 'role:admin|receptionist'])->group(function () {
    Route::apiResource('patients', PatientController::class);
});
```

---

4. SQLite as Default Database (Production Blocker)

What was found: `.env.example` sets `DB_CONNECTION=sqlite` with no MySQL/PostgreSQL config. SQLite has no connection pooling, no concurrent writes, and no production support.

Fix: Change `.env.example` to use MySQL as default:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=cms_clinic
DB_USERNAME=cms_user
DB_PASSWORD=secret

# Keep SQLite only for testing (already correct in phpunit.xml)
```

---

5. `APP_DEBUG=true` in `.env.example` (Security Vulnerability)

What was found: `.env.example` ships with `APP_DEBUG=true` and `APP_ENV=local`. If a developer forgets to change this in production, stack traces (with DB credentials, file paths, and env vars) are leaked to API consumers.

Fix:

```
APP_ENV=production
APP_DEBUG=false
```

Create a separate `.env.example.local` for dev convenience if needed. Add a pre-deploy check in your CI pipeline:

```
# .github/workflows/deploy.yml
- name: Assert debug is off
  run: |
    grep -q "APP_DEBUG=false" .env || exit 1
```

🟠 HIGH — Fix Before Going Live

6. No Service Layer (Fat Controllers)

What was found: With no `app/Services/` directory in the model breakdown, business logic is almost certainly living in controllers. This makes unit testing impossible and creates spaghetti as the system grows.

Fix: Extract a Service layer. Example for appointments:

```
app/
├── Http/
│   └── Controllers/
│       └── Api/V1/
│           └── AppointmentController.php   ← thin: validate, delegate, respond
├── Services/
│   └── AppointmentService.php              ← business logic lives here
├── Repositories/
│   └── AppointmentRepository.php           ← DB queries live here
└── DTOs/
    └── CreateAppointmentDTO.php            ← typed data transfer
```

app/Services/AppointmentService.php：

```php
<?php

namespace App\Services;

use App\DTOs\CreateAppointmentDTO;
use App\Models\Appointment;
use App\Repositories\AppointmentRepository;
use Illuminate\Support\Facades\DB;

class AppointmentService
{
    public function __construct(
        private AppointmentRepository $repo
    ) {}

    public function create(CreateAppointmentDTO $dto): Appointment
    {
        return DB::transaction(function () use ($dto) {
```

```
            // Conflict check
            if ($this->repo->hasConflict($dto->doctorId, $dto->scheduledAt)) {
                throw new \App\Exceptions\AppointmentConflictException();
            }
            return $this->repo->create($dto);
        });
    }
}
```

app/Http/Controllers/Api/V1/AppointmentController.php：

```
public function store(StoreAppointmentRequest $request): JsonResponse
{
    $appointment = $this->appointmentService->create(
        CreateAppointmentDTO::fromRequest($request)
    );

    return (new AppointmentResource($appointment))
        ->response()
        ->setStatusCode(201);
}
```

---

7. No API Versioning

What was found: No API version prefix. Once clients are in production and you need to change a response shape, you have no upgrade path.

Fix: Wrap all routes under `/api/v1/`：

routes/api.php：

```
Route::prefix('v1')->name('api.v1.')->group(function () {
    // Auth
    Route::post('auth/login',   [AuthController::class, 'login'])->name('auth.log
    Route::post('auth/refresh', [AuthController::class, 'refresh'])->name('auth.r

    Route::middleware('auth:api')->group(function () {
        Route::post('auth/logout', [AuthController::class, 'logout']);

        // Doctor-only
        Route::middleware('role:doctor')->group(function () {
            Route::apiResource('appointments', AppointmentController::class);
            Route::apiResource('prescriptions', PrescriptionController::class);
            Route::apiResource('schedules', DoctorScheduleController::class);
```

```
        });

        // Admin-only
        Route::middleware('role:admin')->group(function () {
            Route::apiResource('clinics', ClinicController::class);
            Route::apiResource('users', UserController::class);
        });

        // Shared
        Route::apiResource('patients', PatientController::class);
    });
});
```

---

8. No API Resource Transformers

What was found: From the model structure, the API is likely returning raw Eloquent models. This exposes internal field names, password hashes if not guarded correctly, and creates coupling between your DB schema and your API contract.

Fix: Add `JsonResource` classes for every model:

```
php artisan make:resource AppointmentResource
php artisan make:resource PatientResource
php artisan make:resource UserResource
```

app/Http/Resources/AppointmentResource.php：

```
class AppointmentResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id'           => $this->id,
            'status'       => $this->status,
            'scheduled_at' => $this->scheduled_at->toIso8601String(),
            'doctor'       => new UserResource($this->whenLoaded('doctor')),
            'patient'      => new PatientResource($this->whenLoaded('patient')),
            'prescription' => new PrescriptionResource($this->whenLoaded('prescri
            'created_at'   => $this->created_at->toIso8601String(),
        ];
    }
}
```

---

9. No Form Request Validation

What was found: No `app/Http/Requests/` directory evident. Validation is likely inlined in controllers or missing entirely — exposing the API to malformed inputs.

Fix:

```
php artisan make:request StoreAppointmentRequest
php artisan make:request StorePatientRequest
```

app/Http/Requests/StoreAppointmentRequest.php：

```
class StoreAppointmentRequest extends FormRequest
{
    public function authorize(): bool
    {
        return $this->user()->can('manage appointments');
    }

    public function rules(): array
    {
        return [
            'patient_id'   => ['required', 'integer', 'exists:patients,id'],
            'scheduled_at' => ['required', 'date', 'after:now'],
            'type'         => ['required', 'string', 'in:consultation,follow_up,e
            'notes'        => ['nullable', 'string', 'max:1000'],
        ];
    }
}
```

---

10. No Centralized Error Handling

What was found: No custom exception handler beyond Laravel's default. API consumers receive HTML error pages for unhandled exceptions in some versions, and inconsistent JSON error shapes.

Fix: In `bootstrap/app.php` (Laravel 11+), register custom exception rendering:

```
->withExceptions(function (Exceptions $exceptions) {
    $exceptions->render(function (\App\Exceptions\AppointmentConflictException $e
        return response()->json([
            'error'   => 'appointment_conflict',
            'message' => 'The doctor already has an appointment at this time.',
```

```
        ], 409);
    });

    $exceptions->render(function (\Illuminate\Auth\AuthenticationException $e, Re
        return response()->json(['error' => 'unauthenticated', 'message' => 'Toke
    });

    $exceptions->render(function (\Illuminate\Validation\ValidationException $e,
        return response()->json([
            'error'   => 'validation_failed',
            'errors'  => $e->errors(),
        ], 422);
    });

    $exceptions->render(function (\Illuminate\Auth\Access\AuthorizationException
        return response()->json(['error' => 'forbidden', 'message' => 'Insufficie
    });
})
```

---

11. Cache & Queue on Database (Performance)

What was found: `.env.example` sets `CACHE_STORE=database` and `QUEUE_CONNECTION=database`. This makes your background jobs compete with your main DB traffic, and cache lookups do full table scans.

Fix — switch to Redis:

```
CACHE_STORE=redis
QUEUE_CONNECTION=redis
SESSION_DRIVER=redis
REDIS_CLIENT=phpredis
REDIS_HOST=redis       # service name in Docker
REDIS_PORT=6379
```

Add Redis to `config/database.php` cache config:

```
'redis' => [
    'client' => env('REDIS_CLIENT', 'phpredis'),
    'default' => [
        'host'     => env('REDIS_HOST', '127.0.0.1'),
        'port'     => env('REDIS_PORT', '6379'),
        'database' => env('REDIS_DB', '0'),
    ],
    'cache' => [
```

```
        'host'     => env('REDIS_HOST', '127.0.0.1'),
        'port'     => env('REDIS_PORT', '6379'),
        'database' => env('REDIS_CACHE_DB', '1'),
    ],
],
```

🟡 MEDIUM — Architecture & Quality

---

12. No Rate Limiting

Add to `app/Providers/AppServiceProvider.php`:

```
use Illuminate\Cache\RateLimiting\Limit;
use Illuminate\Support\Facades\RateLimiter;

RateLimiter::for('api', function (Request $request) {
    return Limit::perMinute(60)->by($request->user()?->id ?: $request->ip());
});

RateLimiter::for('auth', function (Request $request) {
    return Limit::perMinute(5)->by($request->ip()); // brute force protection
});
```

In routes:

```
Route::middleware(['throttle:auth'])->group(function () {
    Route::post('auth/login', [AuthController::class, 'login']);
});

Route::middleware(['auth:api', 'throttle:api'])->group(function () {
    // ...all protected routes
});
```

---

13. No Standardized API Response Shape

Create a `ApiResponse` trait for consistent structure:

```
// app/Traits/ApiResponse.php
trait ApiResponse
```

```
{
    protected function success(mixed $data, string $message = 'OK', int $status =
    {
        return response()->json([
            'success' => true,
            'message' => $message,
            'data'    => $data,
        ], $status);
    }

    protected function error(string $message, int $status, array $errors = []): J
    {
        return response()->json([
            'success' => false,
            'message' => $message,
            'errors'  => $errors,
        ], $status);
    }

    protected function paginated(LengthAwarePaginator $paginator, string $resourc
    {
        return response()->json([
            'success' => true,
            'data'    => $resourceClass::collection($paginator),
            'meta'    => [
                'current_page' => $paginator->currentPage(),
                'last_page'    => $paginator->lastPage(),
                'per_page'     => $paginator->perPage(),
                'total'        => $paginator->total(),
            ],
        ]);
    }
}
```

---

14. Dev Artifacts Committed to Repo

What was found: `_ide_helper_models.php` and `.phpstorm.meta.php` (dev-only IDE helper files) are committed to the repo. These should never be in version control.

Add to `.gitignore`:

```
# IDE helpers (generated, not committed)
_ide_helper.php
_ide_helper_models.php
```

```
.phpstorm.meta.php
```

---

15. Missing Structural Directories

The following directories need to be created:

```
app/
├── DTOs/                    # Data Transfer Objects (typed input structs)
├── Enums/                   # ✅ Gender already here — add AppointmentStatus, Us
├── Exceptions/              # Custom exception classes
├── Http/
│   ├── Controllers/
│   │   └── Api/
│   │       └── V1/          # Versioned controllers
│   ├── Middleware/          # Custom middleware
│   ├── Requests/            # Form requests for all endpoints
│   └── Resources/           # API resource transformers
├── Repositories/            # DB query abstraction
├── Services/                # Business logic
└── Traits/                  # Reusable traits (ApiResponse, etc.)
```

---

16. No Swagger / OpenAPI Documentation

```
composer require darkaonline/l5-swagger
php artisan vendor:publish --provider "L5Swagger\L5SwaggerServiceProvider"
```

Annotate controllers:

```
/**
 * @OA\Post(
 *     path="/api/v1/appointments",
 *     tags={"Appointments"},
 *     security={{"bearerAuth":{}}},
 *     @OA\RequestBody(required=true, @OA\JsonContent(ref="#/components/schemas/S
 *     @OA\Response(response=201, description="Appointment created", @OA\JsonCont
 *     @OA\Response(response=409, description="Appointment conflict")
 * )
 */
```

🐋 Docker — Full Implementation

Dockerfile

```
# ─── Stage 1: Composer dependencies ──────────────────────────────
FROM composer:2.8 AS composer-build
WORKDIR /app
COPY composer.json composer.lock ./
RUN composer install \
    --no-dev \
    --no-scripts \
    --no-autoloader \
    --ignore-platform-reqs

COPY . .
RUN composer dump-autoload --optimize --no-dev

# ─── Stage 2: Node/Vite assets ──────────────────────────────────
FROM node:22-alpine AS node-build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --ignore-scripts
COPY . .
RUN npm run build

# ─── Stage 3: Production image ──────────────────────────────────
FROM php:8.3-fpm-alpine AS production

# Install system dependencies
RUN apk add --no-cache \
    nginx \
    supervisor \
    curl \
    libpng-dev \
    libzip-dev \
    oniguruma-dev \
    icu-dev \
    && docker-php-ext-install \
        pdo_mysql \
        mbstring \
        zip \
        gd \
        intl \
        opcache \
```

```
    && apk add --no-cache --virtual .build-deps $PHPIZE_DEPS \
    && pecl install redis \
    && docker-php-ext-enable redis \
    && apk del .build-deps

# PHP-FPM + OPcache config
COPY docker/php/php.ini    /usr/local/etc/php/conf.d/app.ini
COPY docker/php/opcache.ini /usr/local/etc/php/conf.d/opcache.ini

# Nginx config
COPY docker/nginx/default.conf /etc/nginx/http.d/default.conf

# Supervisor config (manages php-fpm, nginx, queue worker)
COPY docker/supervisor/supervisord.conf /etc/supervisor/conf.d/supervisord.conf

WORKDIR /var/www/html

# Copy app code
COPY --from=composer-build /app /var/www/html
COPY --from=node-build      /app/public/build /var/www/html/public/build

# Fix permissions
RUN chown -R www-data:www-data /var/www/html/storage /var/www/html/bootstrap/cach
    && chmod -R 775 /var/www/html/storage /var/www/html/bootstrap/cache

EXPOSE 80

ENTRYPOINT ["/var/www/html/docker/entrypoint.sh"]
```

---

docker-compose.yml

```
version: "3.9"

services:

  # ── Application (PHP-FPM + Nginx) ──────────────────────────────
  app:
    build:
      context: .
      target: production
    container_name: cms_app
    restart: unless-stopped
    environment:
      APP_ENV:            ${APP_ENV:-production}
```

```
    APP_KEY:            ${APP_KEY}
    APP_DEBUG:          "false"
    APP_URL:            ${APP_URL:-http://localhost}
    DB_CONNECTION:      mysql
    DB_HOST:            db
    DB_PORT:            3306
    DB_DATABASE:        ${DB_DATABASE:-cms_clinic}
    DB_USERNAME:        ${DB_USERNAME:-cms_user}
    DB_PASSWORD:        ${DB_PASSWORD}
    REDIS_HOST:         redis
    REDIS_PORT:         6379
    CACHE_STORE:        redis
    QUEUE_CONNECTION:   redis
    SESSION_DRIVER:     redis
    JWT_SECRET:         ${JWT_SECRET}
    JWT_BLACKLIST_ENABLED: "true"
  ports:
    - "8080:80"
  volumes:
    - app_storage:/var/www/html/storage/app
    - app_logs:/var/www/html/storage/logs
  depends_on:
    db:
      condition: service_healthy
    redis:
      condition: service_healthy
  networks:
    - cms_network
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost/up"]
    interval: 30s
    timeout: 10s
    retries: 3

# ── Queue Worker ──────────────────────────────────────────────────────
queue:
  build:
    context: .
    target: production
  container_name: cms_queue
  restart: unless-stopped
  command: ["php", "artisan", "queue:work", "redis", "--sleep=3", "--tries=3",
  environment:
    APP_ENV:          ${APP_ENV:-production}
    APP_KEY:          ${APP_KEY}
    DB_HOST:          db
    DB_DATABASE:      ${DB_DATABASE:-cms_clinic}
```

```
    DB_USERNAME:      ${DB_USERNAME:-cms_user}
    DB_PASSWORD:      ${DB_PASSWORD}
    REDIS_HOST:       redis
    QUEUE_CONNECTION: redis
  depends_on:
    - app
    - redis
  networks:
    - cms_network

# ── Scheduler ──────────────────────────────────────────────────────────
scheduler:
  build:
    context: .
    target: production
  container_name: cms_scheduler
  restart: unless-stopped
  command: >
    sh -c "while true; do php artisan schedule:run --verbose --no-interaction &
  environment:
    APP_KEY:     ${APP_KEY}
    DB_HOST:     db
    DB_DATABASE: ${DB_DATABASE:-cms_clinic}
    DB_USERNAME: ${DB_USERNAME:-cms_user}
    DB_PASSWORD: ${DB_PASSWORD}
    REDIS_HOST:  redis
  depends_on:
    - app
  networks:
    - cms_network

# ── MySQL ──────────────────────────────────────────────────────────────
db:
  image: mysql:8.4
  container_name: cms_db
  restart: unless-stopped
  environment:
    MYSQL_DATABASE:      ${DB_DATABASE:-cms_clinic}
    MYSQL_USER:          ${DB_USERNAME:-cms_user}
    MYSQL_PASSWORD:      ${DB_PASSWORD}
    MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
  volumes:
    - db_data:/var/lib/mysql
    - ./docker/mysql/my.cnf:/etc/mysql/conf.d/my.cnf:ro
  networks:
    - cms_network
  healthcheck:
```

```
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root", "-p${D
      interval: 10s
      timeout: 5s
      retries: 5

  # ── Redis ──────────────────────────────────────────────────────────────
  redis:
    image: redis:7.4-alpine
    container_name: cms_redis
    restart: unless-stopped
    command: ["redis-server", "--appendonly", "yes", "--requirepass", "${REDIS_PA
    volumes:
      - redis_data:/data
    networks:
      - cms_network
    healthcheck:
      test: ["CMD", "redis-cli", "-a", "${REDIS_PASSWORD}", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5

  # ── Nginx Proxy (optional: use for SSL termination) ────────────────────
  # nginx:
  #   image: nginx:1.27-alpine
  #   ...

volumes:
  db_data:
  redis_data:
  app_storage:
  app_logs:

networks:
  cms_network:
    driver: bridge
```

---

docker/entrypoint.sh

```
#!/bin/sh
set -e

# Wait is handled by Docker healthchecks + depends_on

# Run migrations (idempotent in production)
```

```
php artisan migrate --force

# Cache configuration for performance
php artisan config:cache
php artisan route:cache
php artisan view:cache
php artisan event:cache

# Create storage symlink
php artisan storage:link --quiet || true

# Start supervisord (manages php-fpm + nginx)
exec /usr/bin/supervisord -c /etc/supervisor/conf.d/supervisord.conf
```

---

docker/nginx/default.conf

```
server {
    listen 80;
    server_name _;
    root /var/www/html/public;
    index index.php;

    client_max_body_size 20M;

    # Security headers
    add_header X-Frame-Options           "SAMEORIGIN"   always;
    add_header X-Content-Type-Options    "nosniff"      always;
    add_header X-XSS-Protection          "1; mode=block" always;
    add_header Referrer-Policy           "strict-origin-when-cross-origin" always

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location = /up {
        access_log off;
        return 200 "OK";
        add_header Content-Type text/plain;
    }

    location ~ \.php$ {
        fastcgi_pass            127.0.0.1:9000;
        fastcgi_index           index.php;
        fastcgi_param           SCRIPT_FILENAME $realpath_root$fastcgi_script_nam
```

```
        include                 fastcgi_params;
        fastcgi_read_timeout    300;
        fastcgi_buffer_size     16k;
        fastcgi_buffers         4 16k;
    }

    location ~ /\.(?!well-known).* {
        deny all;
    }
}
```

---

docker/php/opcache.ini

```
[opcache]
opcache.enable=1
opcache.enable_cli=0
opcache.memory_consumption=256
opcache.interned_strings_buffer=16
opcache.max_accelerated_files=20000
opcache.revalidate_freq=0
opcache.validate_timestamps=0   ; MUST be 0 in production
opcache.save_comments=1
opcache.fast_shutdown=1
```

docker/supervisor/supervisord.conf

```
[supervisord]
nodaemon=true
logfile=/var/www/html/storage/logs/supervisord.log

[program:php-fpm]
command=php-fpm --nodaemonize
autostart=true
autorestart=true
stdout_logfile=/dev/stdout
stdout_logfile_maxbytes=0

[program:nginx]
command=nginx -g "daemon off;"
autostart=true
autorestart=true
stdout_logfile=/dev/stdout
stdout_logfile_maxbytes=0
```

🟢 LOW — Code Quality & Developer Experience

17. Add CI/CD with GitHub Actions

`.github/workflows/ci.yml`：

```
name: CI

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest

    services:
      mysql:
        image: mysql:8.4
        env:
          MYSQL_ROOT_PASSWORD: secret
          MYSQL_DATABASE: cms_test
        ports: ["3306:3306"]
        options: --health-cmd="mysqladmin ping" --health-interval=10s

    steps:
      - uses: actions/checkout@v4

      - name: Setup PHP
        uses: shivammathur/setup-php@v2
        with:
          php-version: '8.3'
          extensions: pdo_mysql, redis, gd
          coverage: xdebug

      - name: Install dependencies
        run: composer install --no-interaction --prefer-dist

      - name: Copy env
        run: cp .env.example .env && php artisan key:generate

      - name: Assert debug off in example
        run: grep -q "APP_DEBUG=false" .env.example
```

```
      - name: Run Pint (linter)
        run: ./vendor/bin/pint --test

      - name: Run PHPUnit with coverage
        run: php artisan test --coverage --min=80
        env:
          DB_CONNECTION: mysql
          DB_HOST: 127.0.0.1
          DB_DATABASE: cms_test
          DB_USERNAME: root
          DB_PASSWORD: secret
```

---

18. Improve `.gitignore`

```
# Environment
.env
.env.*
!.env.example

# Dev artifacts
_ide_helper.php
_ide_helper_models.php
.phpstorm.meta.php

# Build artifacts
/public/build
/public/hot

# Composer / Node
/vendor/
/node_modules/

# Laravel generated
/storage/app/public
/storage/framework/cache/data
/storage/framework/sessions
/storage/framework/views
/storage/logs

# Docker volumes
docker/mysql/data/
```

---

19. Add Laravel Telescope (Dev Only)

```
composer require laravel/telescope --dev
php artisan telescope:install
php artisan migrate
```

In `AppServiceProvider::register()`:

```
if ($this->app->environment('local')) {
    $this->app->register(\Laravel\Telescope\TelescopeServiceProvider::class);
}
```

Implementation Priority Roadmap

Priority	Issue	Effort	Impact
🔴 1	Remove Passport/Sanctum, keep JWT only	2h	Unblocks production
🔴 2	Delete custom RefreshToken, use JWT blacklist	1h	Security
🔴 3	Add RBAC with spatie/laravel-permission	4h	Authorization
🔴 4	Switch .env.example to MySQL, APP_DEBUG=false	30min	Security
🟠 5	Introduce Service/Repository layer	8h	Maintainability
🟠 6	Add API versioning (/api/v1/)	2h	Future-proofing
🟠 7	Add Form Requests + API Resources	6h	Validation + API contract
🟠 8	Centralized error handler	2h	API consistency
🟠 9	Switch Cache/Queue to Redis	1h	Performance
🟠 10	Rate limiting	1h	Security
🐋 11	Docker + Docker Compose (see above)	4h	Deployment
🟡 12	Standardized API response trait	2h	DX

🟡 13	Remove IDE helpers from git	15min	Hygiene
🟡 14	Swagger/OpenAPI docs	4h	API documentation
🟢 15	GitHub Actions CI/CD	2h	Automation

Total estimated effort: ~40–45 hours of focused refactoring to reach full production readiness.

Positive Notes (What's Already Good)

- ✅ PHP 8.3 + Laravel 13 — latest versions
- ✅ `App\Enums\Gender` — proper PHP 8.1 enum usage
- ✅ IDE helper generation configured (just don't commit the output)
- ✅ PHPUnit configured correctly with in-memory SQLite for tests
- ✅ Laravel Pint included — code style tooling is there
- ✅ PSR-4 autoloading correctly configured
- ✅ Clear domain model: Clinic → Doctor/User → Patient → Appointment → Prescription
- ✅ `DoctorSchedule` model shows scheduling is already modeled